

COMPUTACIÓN DE ALTAS PRESTACIONES

PRÁCTICA 3: Spark y Kubernetes

AUTORES

CARLOS GARCÍA SANTA

CELIA MARTÍN ÁLVAREZ

GRUPO 1461 : PAREJA 03

INGENIERÍA INFORMÁTICA

ESCUELA POLITÉCNICA SUPERIOR

UNIVERSIDAD AUTÓNOMA DE MADRID



//2024

Índice

PRÁCTICA 3: Spark y Kubernetes.....	1
Parte 1.....	2
Ejercicios básicos.....	2
Ejercicios intermedios.....	7
Parte 2.....	11
Ejercicios básicos.....	11
Ejercicios intermedios.....	14

Parte 1

Ejercicios básicos

- **(A.1) (Hadoop vs Spark) Enumere las principales diferencias entre Apache Spark y Apache Hadoop, especialmente explicando el concepto "disk/storage oriented".**
 - Apache Spark se puede ejecutar de forma independiente o en Apache Hadoop, Apache Mesos, Kubernetes, la nube y distintas fuentes de datos.
 - Apache Spark no tiene su propio sistema de archivos nativo.
 - Spark almacena y procesa los datos en la memoria interna.
 - Apache Spark utiliza el almacenamiento en memoria caché.
 - Apache Hadoop procesa y almacena datos en un almacenamiento externo.
 - Apache Hadoop permite agrupar varios equipos para analizar conjuntos de datos enormes en paralelo con mayor rapidez.
 - Apache Hadoop provee un software para el almacenamiento distribuido y procesamiento de grandes datos.
- **(A.2) (Instalación) ¿Cómo se instala Apache Spark? Describa los comandos así como las dependencias de paquetes.**

En primer lugar hay que comprobar que está instalado en nuestro ordenador Java, ya que Apache Spark lo requiere.

```
!java -version
```

Si no está instalado, ponemos el siguiente comando para instalarlo.

```
!apt-get update
```

```
!apt-get install openjdk-8-jdk-headless -qq
```

Una vez instalado Java descargamos Spark desde la página web, usando wget más el link de la página web.

```
!wget  
https://downloads.apache.org/spark/spark-3.5.3/spark-3.5.3-bin-hadoop3.tgz
```

Ahora extraemos el archivo del fichero .tgz.

```
!tar -xf spark-3.5.3-bin-hadoop3.tgz
```

Para poder usar el entorno en Python, instalamos PySpark

```
!pip install pyspark
```

Después configuramos la variables de entorno:

```
import os

os.environ["JAVA_HOME"] = "/usr/lib/jvm/java-8-openjdk-amd64"

os.environ["SPARK_HOME"] = f"/content/spark-3.5.3-bin-hadoop3"
```

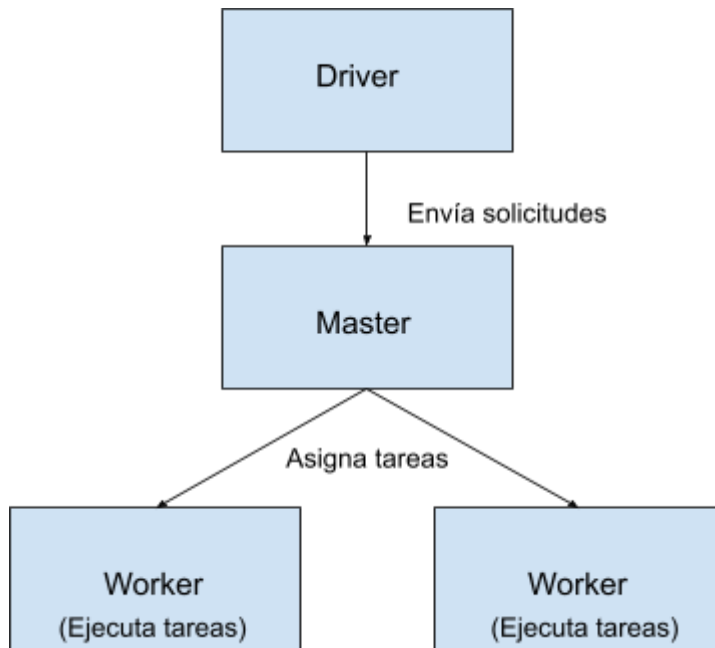
Si queremos verificar que se ha instalado spark, ponemos:

```
!spark-shell
```

- **(A.3) (Cluster I) ¿Qué elementos componen un cluster de Apache Spark? Elabore un diagrama que indique cómo se comunican *driver*, *master* y *worker*.**

Los elementos que componen un cluster son:

- **Driver:** Es el que inicia la aplicación de Spark. Se encarga de dividir y distribuir las tareas entre los nodos Workers a través del Master. Una vez terminada las tareas, recoge los resultados de cada ejecución de los nodos Workers.
- **Master:** Es el que se encarga de administrar dentro del cluster. Estos asignan los recursos a los Workers en función de lo que solicite el Driver. Suele haber un Master por cluster.
- **Worker:** Es el que se encarga de realizar las tareas que se le ha asignado. Cada Worker tiene hilos y memoria asignada para poder realizar tareas en paralelo. Dentro de los Workers puede haber Executors que son procesos que se encargan de ejecutar tareas individuales y almacenar los datos en la memoria para acelerar el procesamiento.



- **(A.4) (Homogeneidad I) ¿Qué es un RDD? ¿Son los RDDs homogéneos (todos los elementos son del mismo tipo)? Escriba un fragmento de código que compruebe o rechace esta afirmación.**

Un RDD es una estructura fundamental en Apache Spark. Es una colección de elementos distribuidos en el cluster, permitiendo el procesamiento distribuido y paralelo, con grandes volúmenes de datos.

No necesariamente todos los RDDs son homogéneos. Apache Spark puede almacenar distintos tipos de RDDs, sólo depende de los datos iniciales y de cómo se haya construido el RDD.

Código:

```
from pyspark import SparkContext

# Inicializar el contexto de Spark

sc = SparkContext

# Probamos distintos tipos de elementos (int, str, double, dic, list)

rdd = sc.parallelize([1, "hola", 3.5, [1, 2, 3]])

def homogeneo(rdd):
```

```

types = rdd.map(type).distinct().collect()

if len(types) == 1:

    print("Todos los elementos son del mismo tipo")

else:

    print("El RDD contiene elementos de diferentes tipos")

# Comprobar homogeneidad

homogeneo(rdd)

```

Resultado:

El RDD contiene elementos de diferentes tipos

- **(A.5) (Particiones I) ¿Qué es el número de particiones de un RDD? ¿Cómo puedo crear un RDD con un cierto número fijo de particiones? ¿Puedo conocer el número de particiones de un RDD?**

El número de particiones de un RDD son las divisiones de los datos que realiza Spark para que se pueda paralelizar.

Para crear un RDD con un cierto número fijo de particiones se puede especificar a la hora de paralelizar.

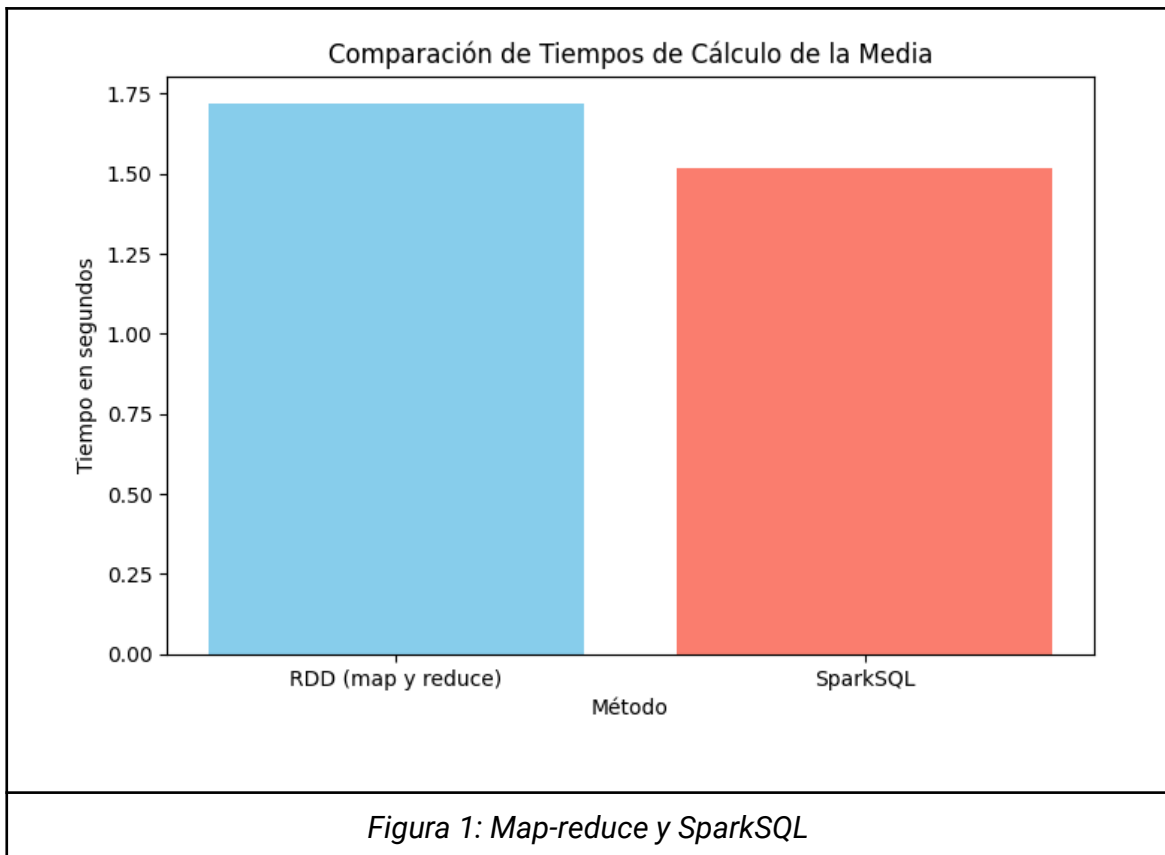
rdd = sc.parallelize(data, numSlices=4)

En este ejemplo vemos que se va a realizar una partición de 4 bloques.

El número de particiones de un RDD se puede conocer usando el método **getNumPartitions()**.

- **(A.6) (SparkSQL vs RDD) El API de SparkSQL utiliza RDDs para realizar todos sus cálculos. ¿Puedo obtener alguna columna en formato RDD? Calcule la media del RDD (utilizando map y reduce y utilizando la función específica para la media) y utilizando el API de SparkSQL y compare tiempos entre ambas operaciones.**

Sí, se puede obtener alguna columna en formato RDD, para ello habrá que convertir el DataFrame en un RDD y después mapear para obtener los valores de la columna.



Podemos observar que al usar map y reduce el tiempo es mayor que al usar SparkSQL, esto se puede deber a las optimizaciones internas que tiene Spark.

- **(A.7) (Carga y creación de datos I) Consulte la documentación y resuma los principales tipos de carga de datos de RDDs. De igual modo, realice el mismo ejercicio para el API de SparkSQL.**

Se pueden cargar datos desde ficheros de texto. Archivos locales o en HDFS. Para realizar esto hay que poner el siguiente comando: **sc.textFile(path)**. Esto lo que hace es leer el fichero y devuelve un RDD de strings donde cada línea corresponde a un elemento del RDD.

También se pueden cargar datos usando ficheros de tipo json. Para ello habría que usar SparkSQL que tiene funciones para poder leer json. Un ejemplo puede ser el siguiente: **rdd = sc.textFile(path).map(parseJson)**. Mapeamos a json para poder trabajar directamente con el RDD.

Otra forma es usando bases de datos. Usamos bases de datos relacionadas donde leemos sus tablas y las convertimos en RDDs. El resultado obtenido es un DataFrame pero se puede convertir a RDD. Ejemplo: **val rdd = sqlContext.read.jdbc("jdbc:mysql://localhost/db", "table_name", connectionProperties)**

Se pueden cargar datos desde colecciones locales. Para ello usamos **rdd = sc.parallelize(List(1, 2, 3, 4))**.

Los RDDs también pueden ser creados a partir de otros RDDs usando transformaciones.

Otra forma puede ser usando particionamiento. Ya que si los datos están distribuidos en particiones, podemos especificar el número de particiones que vamos a hacer. Ejemplo: **rdd = sc.textFile("path, numPartitions = 10).**

Para SparkSQL hay otras formas de cargar los datos. Podemos hacerlo usando **spark.read.csv(path)** para poder leer archivos csv, que los carga como DataFrames.

Para distintos tipos de archivos siempre hay que poner **spark.read** y luego lo que sigue varía dependiendo del archivo que quieras leer.

Se pueden cargar datos desde colecciones de memoria. **spark.createDataFrame(data)**, esto lo que hace es crear DataFrames a partir de colecciones locales con un esquema definido.

- **(A.8) (Guardado de datos I) Consulte la documentación y resuma los principales tipos de guardado de datos de RDDs. De igual modo, realice el mismo ejercicio para el API de SparkSQL.**

En RDDs se pueden guardar de diversas formas los datos. Cuando los datos se quieren guardar en archivos de texto usamos **rdd.saveAsTextFile(path)**, donde cada elemento del RDD será una línea en el fichero de texto.

También se pueden guardar en archivos de objetos, usando **rdd.saveAsObjectFile(path)** permitiendo una serialización eficiente de los datos.

En SparkSQL, podemos guardar los datos de distintas formas. Al igual que al cargar datos para guardar en distintos tipos de archivos se debe de poner **spark.write** y luego se especifica el tipo de archivos que vamos a usar.

Ejercicios intermedios

- **(B.1) (Cluster II) Un problema frecuente de los programas en Spark es el uso de memoria en el *driver*. ¿Cómo puede conocer este parámetros en el driver desde el API de PySpark? De igual modo, la memoria de los workers puede afectar al tamaño máximo de las particiones. ¿Cómo puedo conocer este parámetros en el driver desde el API de PySpark?**

Para poder conocer el uso de memoria en el driver desde la API de PySpark, lo que hay que hacer es primero sacar el contexto de spark. Una vez sabemos el contexto hacemos el siguiente comando **sc.getConf().get("spark.driver.memory")**, esto nos devolverá la cantidad de memoria que el driver podrá usar.

Para saber la memoria que los worker tienen desde el API de PySpark , usamos el siguiente comando, una vez tenemos el contexto de spark:

`sc.getConf().get("spark.executor.memory")`, esto devuelve la cantidad de memoria que cada worker puede usar.

- **(B.2) (Cluster III) Investigue maneras de conocer cuántos workers están disponibles. ¿Cree que este parámetro debe relacionarse con los números de particiones de los RDDs?**

- Se puede acceder al número de ejecutores. Para ello ponemos `spark.sparkContext.statusTracker().getExecutorInfos()` para saber qué ejecutores están activos, por lo que de esta forma podemos saber los workers disponibles
- Se puede saber desde la UI de Spark. Esta monitorea el número de workers a tiempo real. Para ello se puede acceder usando la url `http://<driver-ip>:4040`, donde saldrán los worker que tiene Spark.
- Se puede saber los workers disponibles a través del manager. Con kubernetes se puede saber usando: **kubectrl get nodes**

El número de workers disponibles está relacionado con el número de particiones de los RDDs, ya que en general suele ser múltiplo del número de workers o un número mayor de workers disponibles. Si tienes más particiones que workers, Spark puede distribuir las tareas de manera eficiente entre los workers disponibles. Si tienes menos particiones que workers, algunos podrían estar inactivos.

Esto igualmente no es el único factor a tener en cuenta a la hora de que el número de particiones sea óptimo, ya que también afecta la memoria disponible, la carga de trabajo, y la forma en que los datos deben ser procesados.

- **(B.3) (Homogeneidad II) Escriba una función en PySpark que compruebe si un RDD es homogéneo. ¿Qué pasa si un elemento es None o NaN? ¿Por qué es importante la homogeneidad de los RDDs? Proporcione al menos un motivo centrado en el desarrollo y otro motivo centrado en el rendimiento.**

*La función ya ha sido escrita anteriormente

Si el elemento es None, el RDD no será homogéneo ya que no se puede determinar un tipo consistente para esos elementos. Si el elemento es NaN, no afecta a la homogeneidad del RDD ya que si tu creas un RDD con tipo float y añades un NaN, este actúa como float. Por lo que no afectaría a su homogeneidad. Sin embargo puede causar algún problema a nivel lógico y de procesamiento.

La homogeneidad es importante en los RDDs. En cuanto al desarrollo la homogeneidad facilita operaciones de transformación y análisis sin tener que verificar o manejar constantemente diferentes tipos de datos. Si existen distintos tipos en un RDD es más difícil encontrar los fallos. En cuanto al rendimiento, la homogeneidad del RDD mejora la optimización en las funciones distribuidas. Spark puede realizar operaciones específicas del tipo de elemento usado, reduciendo así la sobrecarga de memoria que se podría producir al intentar procesar distintos tipos de elementos.

- **(B.4) (Particiones II)** En principio, las particiones de un RDD deberían ser equilibradas, pero tras varias operaciones esto puede ser que no sea así. Escriba un código que precisamente cause un desbalanceo de las particiones de un RDD.
Sugerencia: `filter` o `flatMap`.

```
from pyspark import SparkContext

sc = SparkContext.getOrCreate()

# Creamos un RDD con números del 1 al 1000 y 4 particiones

rdd = sc.parallelize(range(1, 1001), numSlices=4)

# Aplicamos el filtro para desbalancear el RDD

rdd_desbalanceado = rdd.filter(lambda x: x <= 10)
```

De esta forma los elementos resultantes no estarán distribuidos uniformemente entre las particiones. Por ejemplo, los números del 1 al 10 podrían estar distribuidos desigualmente, causando que algunas particiones tengan más elementos que otras.

- **(B.5) (Operaciones de shuffling)** Las operaciones de comunicación en Spark son implícitas, pero sabemos que hay varias operaciones que por su naturaleza, implican comunicación (shuffling). De la guía de programación de RDDs, analice las principales transformaciones indicando el nivel de comunicación que se espera en cada una.
 - `map`: nivel de comunicación bajo.
 - `filter`: nivel de comunicación bajo.
 - `flatMap`: nivel de comunicación bajo.
 - `mapPartitions`: nivel de comunicación bajo.
 - `mapPartitionsWithIndex`: nivel de comunicación bajo.
 - `sample`: nivel de comunicación bajo.
 - `union`: nivel de comunicación bajo.
 - `intersection`: nivel de comunicación alto
 - `distinct`: nivel de comunicación alto
 - `groupByKey`: nivel de comunicación alto
 - `reduceByKey`: nivel de comunicación medio-alto
 - `aggregateByKey`: nivel de comunicación medio-alto
 - `sortByKey`: nivel de comunicación alto
 - `join`: nivel de comunicación alto
 - `cogroup`: nivel de comunicación alto
 - `cartesian`: nivel de comunicación muy alto
 - `pipe`: nivel de comunicación bajo
 - `coalesce`: nivel de comunicación medio bajo
 - `repartition`: nivel de comunicación alto
 - `repartitionAndSortWithinPartitions`: nivel de comunicación alto

- **(B.6) (Creación de datos II) Para los métodos Monte Carlo, es importante la generación de números (pseudo-)aleatorios. Investigue los mecanismo de Apache Spark para ello. ¿Cuál sería el potencial problema de utilizar métodos directos de Python para ello? ¿Cómo podríamos garantizar que cada partición es diferente del resto y no simplemente estamos repitiendo exactamente el mismo experimento muchas veces?**

Si usamos los métodos de python para generar número aleatorios corremos el riesgo de contener problemas de repetición, ya que los generadores funcionan de manera secuencial y no están diseñados para un entorno distribuido.

Para poder garantizar que cada partición sea diferente al resto podríamos usar el índice de partición para inicializar un generador de números aleatorios en cada partición de manera única. Otra forma podría ser usar RandomRDDs de Spark. Esto garantiza que cada partición tenga su propia secuencia aleatoria independiente. También se pueden usar librerías de generación de números aleatorios de múltiples núcleos como numpy.

- **(B.7) (Guardado de datos II) El formato parquet es el más común empleado hoy en día para Big Data. Investigue cómo funciona en comparación a formatos como CSV o JSON.**

El formato Parquet se destaca en Big Data y análisis de datos por su estructura en columnas, alta compresión y eficacia en la lectura, especialmente en consultas que se dirigen a columnas concretas en grandes cantidades de datos. A pesar de que puede ser menos rápido para redactar, esto se puede contrarrestar con su desempeño en operaciones de lectura intensiva. Por otro lado, tanto CSV como JSON son formatos más sencillos, apropiados para el intercambio de datos y el almacenamiento temporal, aunque son menos eficaces en cuanto a almacenamiento y rapidez de acceso en operaciones de análisis.

Parte 2

Ejercicios básicos

- (A.0) (Tutorial) Realice el tutorial [Example: Deploying PHP Guestbook application with Redis](#)
- (A.1) (Metadatos, estado y especificación) Extraiga en formato YAML los datos de un despliegue (por ejemplo, del tutorial). Compare los tres campos principales que tiene un objeto en Kubernetes y describa el propósito de cada uno.

Ejemplo:

```
apiVersion: apps/v1

kind: Deployment

metadata:

  name: redis-leader

  labels:

    app: redis

    role: leader

    tier: backend

spec:

  replicas: 1

  selector:

    matchLabels:

      app: redis

  template:

    metadata:

      labels:

        app: redis

        role: leader
```

```

    tier: backend

spec:

  containers:

  - name: leader

    image: "docker.io/redis:6.0.5"

    resources:

      requests:

        cpu: 100m

        memory: 100Mi

    ports:

    - containerPort: 6379

```

En primer lugar está el campo metadata. Esta proporciona información como el nombre y las etiquetas del elemento que vamos a desplegar.

Luego está el campo spec. Este describe la configuración del elemento. En este ejemplo muestra los contenedores que se van a ejecutar, el número de réplicas de los pods.

Otro elemento a tener en cuenta es el campo status, que en este ejemplo no está. Este campo describe el estado actual del elemento. Incluye detalles como la cantidad de réplicas actuales y cuántos pods están disponibles, en ejecución, o fallidos.

- **(A.2) (Despliegues) ¿Cómo puedo conocer los replicasets asociados a un deployment? ¿Y los pods asociados a un deployment? Explique las diferencias entre contenedor, pod, replicaset, job y deployment.**

Para conocer los replicasets asociados al deployment podemos poner por terminal `kubectl describe deployment <nombre-del-deployment>`, de esta forma en el apartado de replicasets aparecerán.

Para conocer los pods asociados a un deployment podemos poner por terminal lo mismo, aunque también se puede poner por terminal `kubectl get pods --selector=app=<nombre-del-deployment>`

- **Contenedor:** es una instancia de una imagen de contenedor que ejecuta una aplicación. Contiene el entorno de ejecución de una aplicación.
- **Pod:** puede contener uno o más contenedores. Agrupa uno o más contenedores que necesitan estar cerca unos de otros, compartir recursos y trabajar juntos. Si en un pod hay más de un contenedor, estos compartirán el mismo almacenamiento, red y especificaciones de red.
- **Replicaset:** Asegura la disponibilidad y la escalabilidad de los Pods, manteniendo el número de réplicas deseadas
- **Job:** es un controlador que se utiliza para ejecutar tareas que deben ejecutarse en un tiempo limitado. Un job ejecuta un pod hasta que ha terminado la tarea y luego finaliza el pod. Se utilizan para ejecutar tareas o trabajos por lotes que no necesitan estar en ejecución continuamente.
- **Deployment:** asegura que se ejecuten una cantidad deseada de réplicas de un pod y permite realizar actualizaciones sin interrupciones. Gestiona los replicasets para asegurar que los pods estén correctamente distribuidos y funcionando.

- **(A.3) (Servicios) Describa los tres tipos principales de servicios en Kubernetes. ¿Cuál es el tipo por defecto? ¿Qué diferencia existe entre port, targetPort y nodePort? ¿Se puede especificar el protocolo de nivel de transporte? Explique los diferentes tipos de servicios, así como la diferencia de un servicio y un Ingress Controller.**

Los tres principales servicios de Kubernetes son **ClusterIP**, **NodePort** y **LoadBalancer**.

Los **ClusterIP** exponen el servicio solo dentro del clúster de Kubernetes. Asignan una dirección IP virtual interna que sólo es accesible por otros servicios y pods en el clúster.

El **NodePort** expone el servicio en un puerto específico de cada nodo del clúster. Este permite realizar accesos internos sin necesidad de tener un controlador de balanceo de carga.

El **LoadBalancer** solicita un balanceador de carga externo de un proveedor de nube. Además asigna un puerto y una dirección IP externa accesible.

El tipo por defecto de los servicios en Kubernetes es el ClusterIP.

Un **port** es el puerto expuesto por el servicio a otros pods o servicios dentro del cluster.

El **targetPort** es el puerto del contenedor en el pod, donde se ejecuta la aplicación.

El **nodePort** es el puerto de cada nodo del cluster que expone el servicio a través de la red externa.

En Kubernetes se puede especificar el protocolo de nivel de transporte. Existen distintos valores: TCP, UDP (aplicaciones que dependen de comunicación sin conexión) y SCTP(aplicaciones específicas).

El **servicio** es un recurso que expone pods seleccionados dentro del clúster o externamente. Este no gestiona el enrutamiento avanzado de solicitudes.

Ingress es un recurso que tiene reglas de enrutamiento como HTTPS para el acceso externo a servicios dentro del clúster. Este requiere un controlador para funcionar.

Ejercicios intermedios

- **(B.0) (Tutorial II) Utilizando los ficheros proporcionados, realice un despliegue de un cluster de Apache Spark en Kubernetes. Responda a las siguientes preguntas:**
 - **Escala el número de workers a 3, mediante `kubectl apply -f` y otro comando.**
 - **Destruye el cluster de Spark, mediante `kubectl delete -f`.**
 - **¿Pueden dos cluster de Spark co-existir? Indique los cambios a realizar.**

Cambiamos worker.yaml:

spec:

replicas: 3

Aplicamos los cambios:

```
kubectl apply -f worker.yaml
```

También podemos emplear comandos directamente: **`kubectl scale deployment spark-worker --replicas=3 -n spark`**

Para destruir los recursos del cluster:

```
kubectl delete -f master.yaml
```

```
kubectl delete -f worker.yaml
```

```
kubectl delete -f master.yaml
```

```
kubectl delete -f worker.yaml
```

Es posible que dos clústeres de Spark coexistan en el mismo clúster de Kubernetes. Para lograrlo hay que realizar cambios en los archivos de despliegue para evitar conflictos de nombres, etiquetas, selectores y puertos.

- **(B.1) (Estado) ¿Puedo especificar el estado (status) de un objeto mediante `kubectl apply`?**

El campo status es administrado automáticamente por Kubernetes y refleja el estado actual del recurso en el cluster, este estado se puede cambiar con el comando indicado y kubernetes se encarga de cambiar el estado del objeto.

- (B.2) (Metadatos y etiquetas) Los metadatos son uno de los atributos más importantes de Kubernetes que controlan como despliegues, templates y servicios interactúan. Compruebe (o refute) que puede poner dentro de labels el nombre de atributo y valor de atributo que desee. Explique como realizar una busqueda de pods según el valor de las etiquetas.

Se pueden modificar las labels dentro de la sintaxis permitida:

Las claves de las etiquetas pueden tener un prefijo opcional (como un dominio) separado por una barra /, y pueden tener hasta 63 caracteres.

Las claves y valores deben comenzar y terminar con una letra alfanumérica ([a-z0-9A-Z]), y pueden contener guiones (-), guiones bajos (_), puntos (.) y letras alfanuméricas.

Para realizar búsquedas de pods en etiquetas se puede utilizar el comando: **kubectl get pods -l etiqueta=valor**

- (B.3) (Alta disponibilidad de despliegues) Una característica interesante de los deployments es la posibilidad de actualizarlo con el menor tiempo de no disponibilidad posible. Esto lo gestiona automáticamente Kubernetes, ¿cómo lo hace para el caso del que cambiamos la imagen del contenedor? Mediante el comando **kubectl set image** pruebe a actualizar una imagen de un despliegue y observe los diferentes replicaset y pods y explique lo que sucede.

1. Kubernetes crea un nuevo ReplicaSet con la nueva especificación (incluyendo la nueva imagen).
2. Se comienzan a crear pods con la nueva imagen mientras se eliminan gradualmente los pods con la imagen antigua.
3. Durante el proceso, Kubernetes asegura que un número mínimo de pods esté siempre disponible, según los parámetros maxUnavailable y maxSurge del Deployment.
4. Una vez que todos los pods están ejecutando la nueva imagen y el antiguo ReplicaSet no tiene pods activos, la actualización se considera completa.